# Welcome

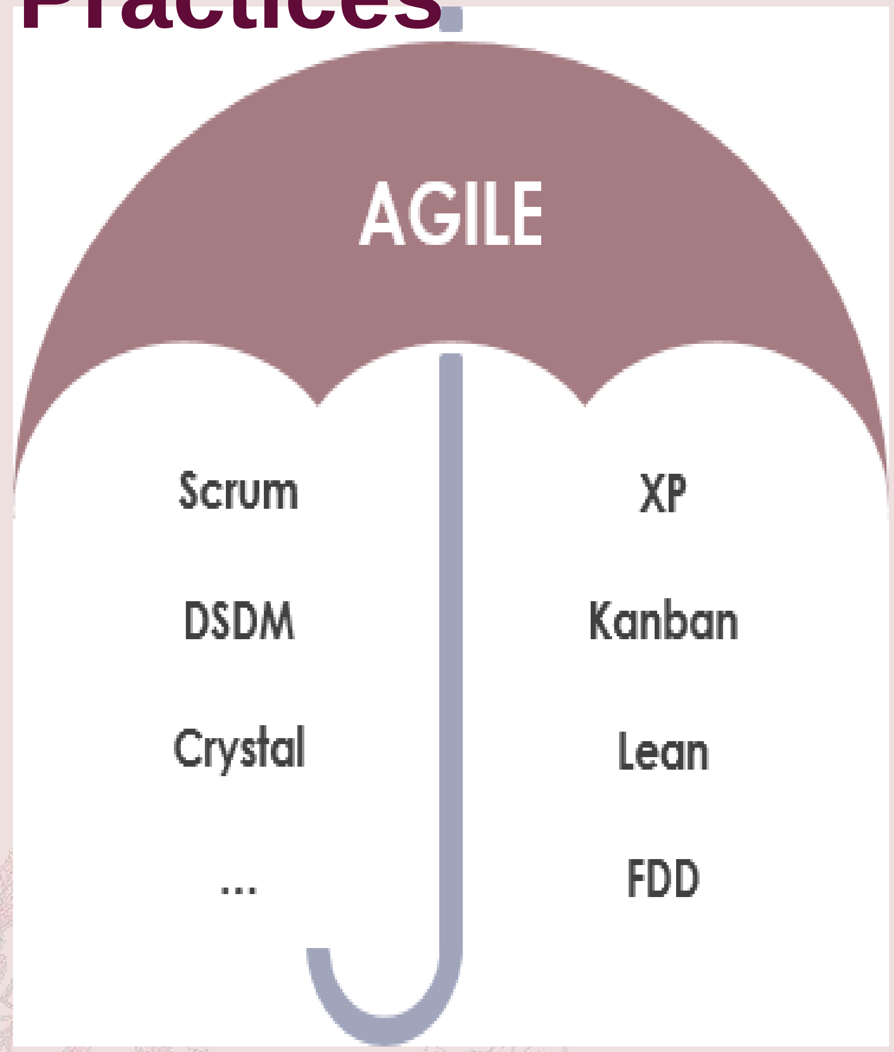
Agile Methodology – Best Practices
by
Girish Godbole

# Agile – The Manifesto

Agile software development emphasizes four core values.

**Individual and team interactions** over processes and tools
**Working software over** comprehensive documentation
**Customer collaboration** over contract negotiation
**Responding to change** over following a plan

# Agile Methods and Practices

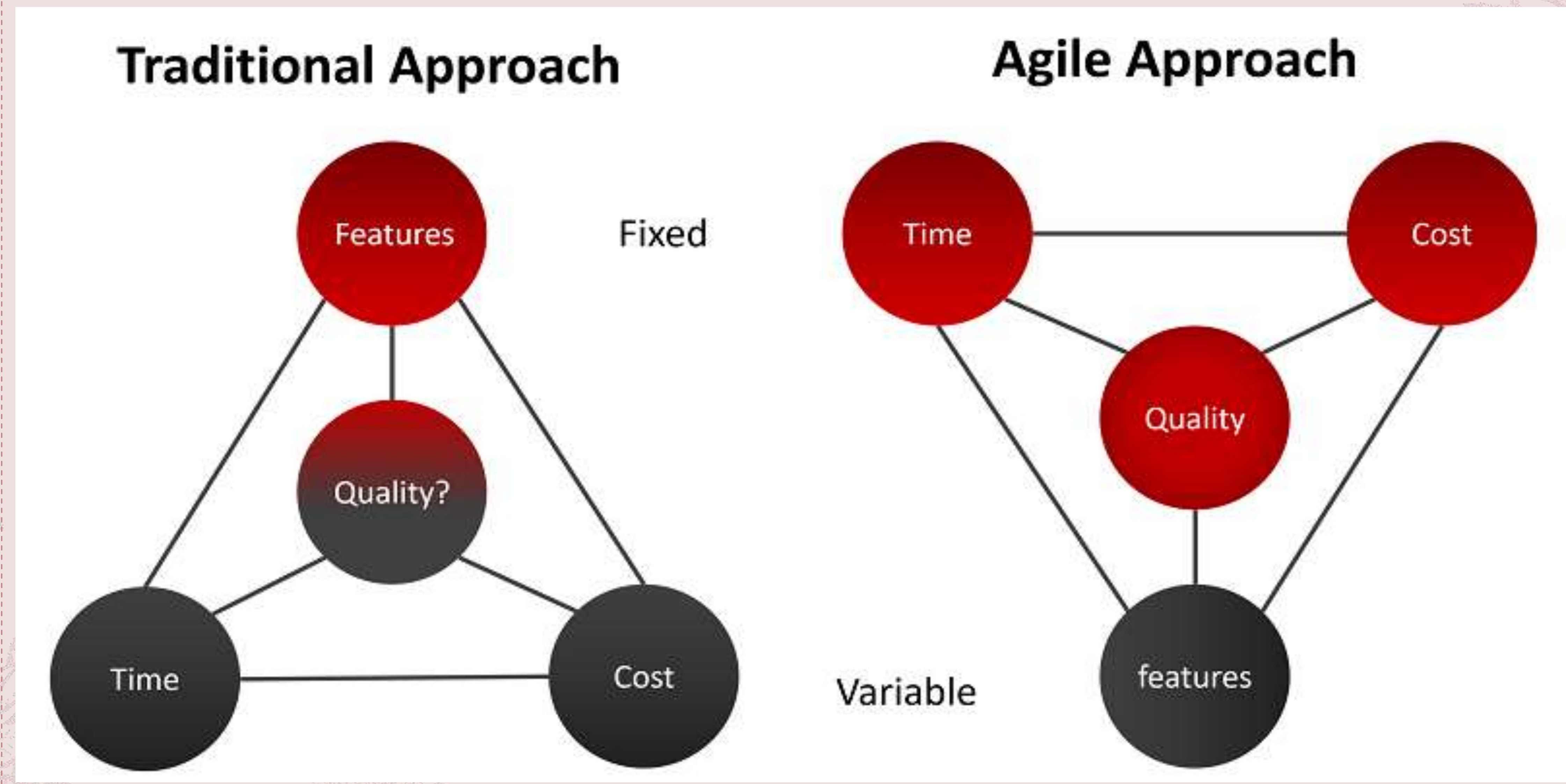



- Scrum
- Crystal Methodologies
- DSDM ( Dynamic Software Development Method )
- Feature driven development (FDD)
- Lean software development
- Extreme Programming (XP)

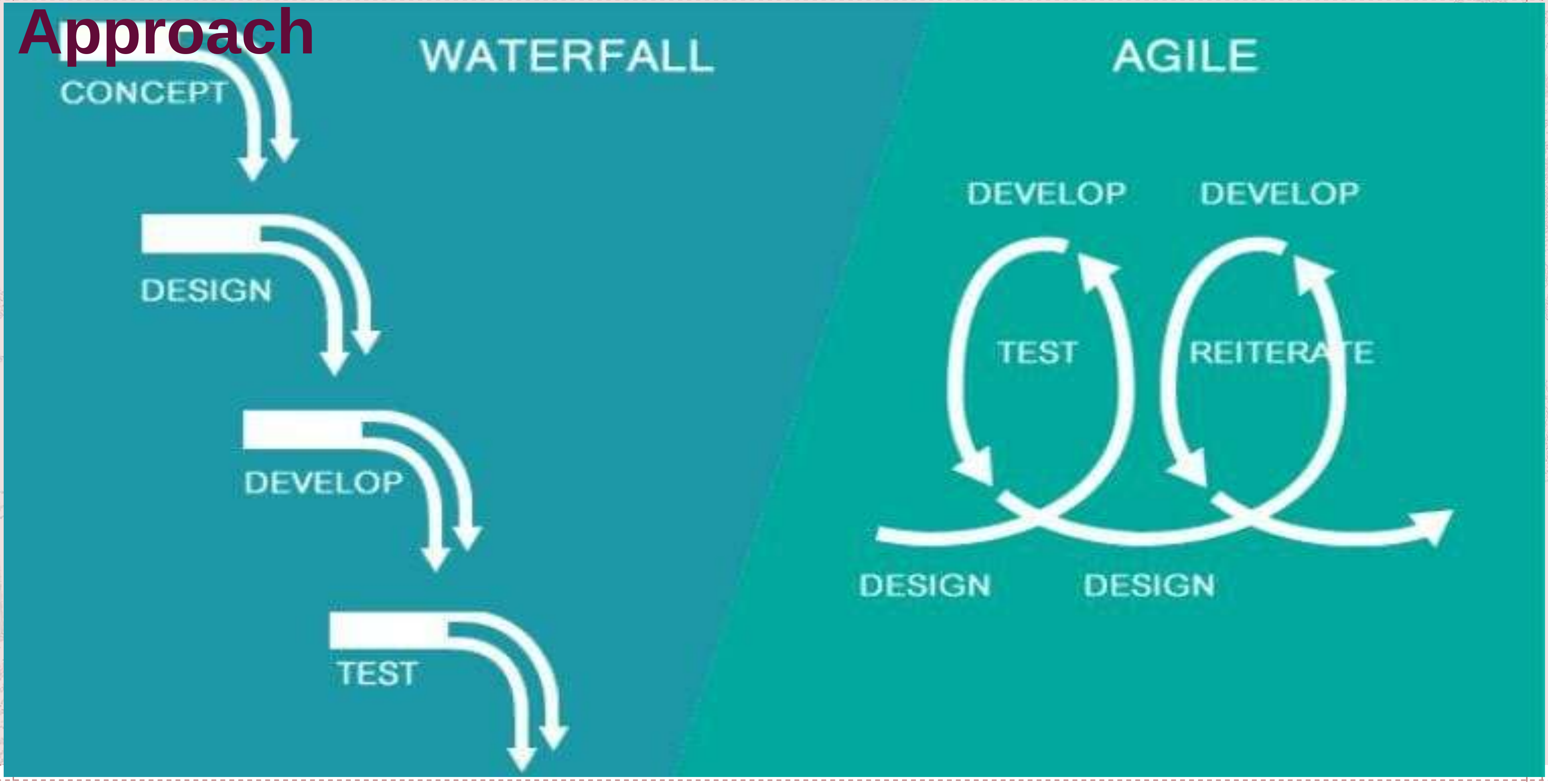
# Agile – The Principles

Highest priority is to satisfy the customer through early and continuous delivery of valuable software.
Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for a shorter timescale.
Businesspeople and developers must work together daily throughout the project.
Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.
The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
Working software is the primary measure of progress.
Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
Continuous attention to technical excellence and good design enhances agility.
Simplicity--the art of maximizing the amount of work not done--is essential.
The best architectures, requirements, and designs emerge from self-organizing teams.
At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

# Agile – Approach



# Traditional vs Agile Approach



# Agile Methods and Practices

**Extreme Programming (XP) –**
Extreme Programming (XP) also emphasizes speed and continuous delivery. Like Scrum, XP enables closely-knit teams to deliver working software increments at frequent intervals, usually every 1-3 weeks. It relies on customers to communicate the most useful features of a software product and developers to work towards implementing that feedback.

The five basic component of Extreme Programming are:
1. Communication
2. Simplicity
3. Feedback
4. Respect
5. Courage

Extreme Programming allow changes in their set timelines.
In Extreme Programming(XP), teamwork for 1-2 weeks only.
Extreme Programming emphasizes strong engineering practices
In Extreme Programming, team must follow a strict priority order or pre-determined priority order
Extreme Programming(XP) can be directly applied to a team. Extreme Programming is also known for its **Ready-to-apply** features.

# Agile Methods and Practices

**Feature-Driven Development (FDD) –**

Feature-Driven Development, or FDD, provides a framework for product development that starts with an overall model and gets progressively more granular. Like other Agile methodologies, FDD aims to deliver working software quickly in a repeatable way. It uses the concept of "just enough design initially" (JEDI) to do so, leveraging two-week increments to run "plan by feature, design by feature, build by feature" iterations.

Organizations that practice Agile like Feature-Driven Development for its feature-centric approach and its scalability.

Five Step Process -

- Developing an overall model
- Building the feature list
- Planning by feature
- Designing by feature
- Building by feature

FDD is amazing for big projects and is quite scalable and prone to get achieve success.

FDD is very effective in helping with complex projects that are in a critical situation.

# Agile Methods and Practices

**Lean Software Development –**

Lean software development is more flexible than Scrum or XP, with fewer strict guidelines, rules, or methods. Lean is based on a set of principles developed to ensure value and efficiency in production in the mid 20th century and has evolved into the software setting. Lean relies on five principles of Lean management:

Identify value
Value stream mapping
Create continuous workflow
Create a pull system
Continuous improvement

Lean particularly emphasizes eliminating waste. In the context of software development, that includes cutting out wasted time and unproductive tasks, efficiently using team resources, giving teams and individuals decision-making authority, and prioritizing only the features of a system that deliver true value.

# Agile Methods and Practices

**Kanban –**

Kanban focuses on helping teams work together more effectively to enable continuous delivery of quality products. Kanban is unique, however, for offering a highly visual method for actively managing the creation of products.

The Kanban Agile methodology relies on six fundamental practices:
Visualize the workflow
Limit work in progress
Manage flow
Make process policies explicit
Implement feedback loops
Improve collaboratively

Kanban achieves these practices using a Kanban board. The Kanban board facilitates the visual approach to Agile using columns to represent work that is To Do, Doing, and Done. This Agile methodology improves collaboration and efficiency and helps define the best possible team workflow.
To Do, In Progress, In Review, and Done (or Completed).
The most important metrics are lead time and cycle time. Each of these metrics measures the average amount of time it takes for tasks to move through the board.

# Agile – Scrum Vs Kanban

| | Scrum | Kanban |
|---|---|---|
| Cadence | Regular fixed length sprints (i.e., 2 weeks) | Continuous flow |
| Release methodology | At the end of each sprint | Continuous delivery |
| Roles | Product owner, scrum master, development team | No required roles |
| Key metrics | Velocity | Lead time, cycle time, WIP |
| Change philosophy | Teams should not make changes during the sprint. | Change can happen at any time |
| Approach | A structured agile approach | Continuous improvement, flexible processes |

# Agile Methods and Practices

**Dynamic Systems Development Method (DSDM)**

The Dynamic Systems Development Method (DSDM) rounds out our list of well-known Agile methodologies. DSDM originated in the 1990s to provide a common industry framework for rapid software delivery. Today, it has matured into a comprehensive

Agile methodology revolves around:
Business needs and value
Active user involvement
Empowered teams
Frequent delivery
Integrated testing
Stakeholder collaboration

The DSDM framework is particularly useful for prioritizing requirements. It also mandates that rework is to be expected, so any development changes must be reversible. DSDM relies on sprints, similar to other Agile methodologies, and is often used in conjunction with approaches like Scrum and XP.

# Agile Methods and Practices

**Crystal -**

The Crystal Agile methodology focuses more on the interactions of the people involved in a project versus the tools and techniques of development. A lightweight model, Crystal emphasizes interaction, people, community, skills, communications, and talents.

Crystal categorizes projects based on three criteria:
Team size
System criticality
Project priorities

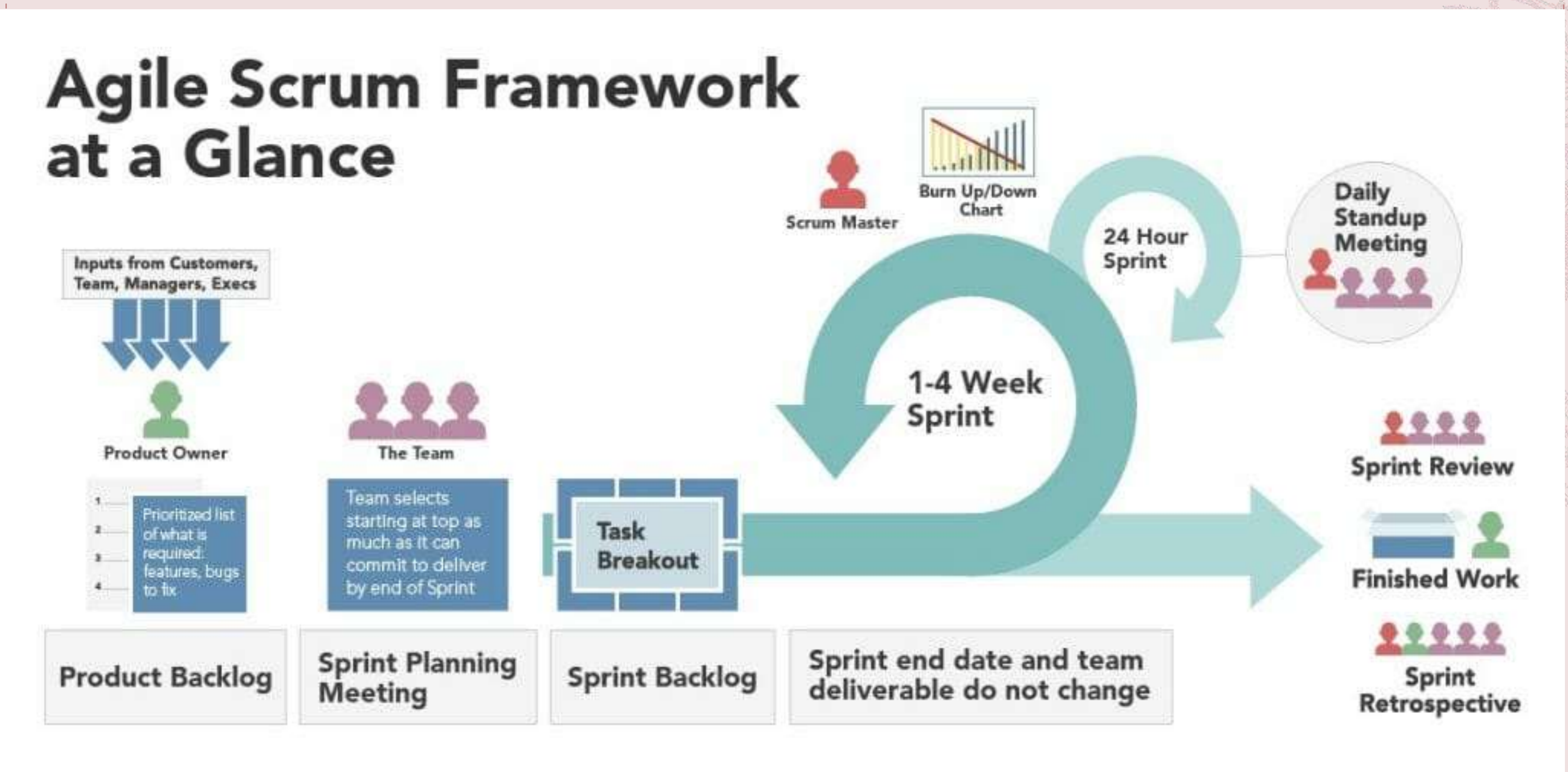
The approach is similar to other Agile methodologies in its attention to early and often delivery of software, high involvement of users, and removal of red tape. Crystal's assertion that every project is unique, however, has led to its reputation as one of the most flexible Agile methodologies.

# Agile Methods and Practices

## Scrum

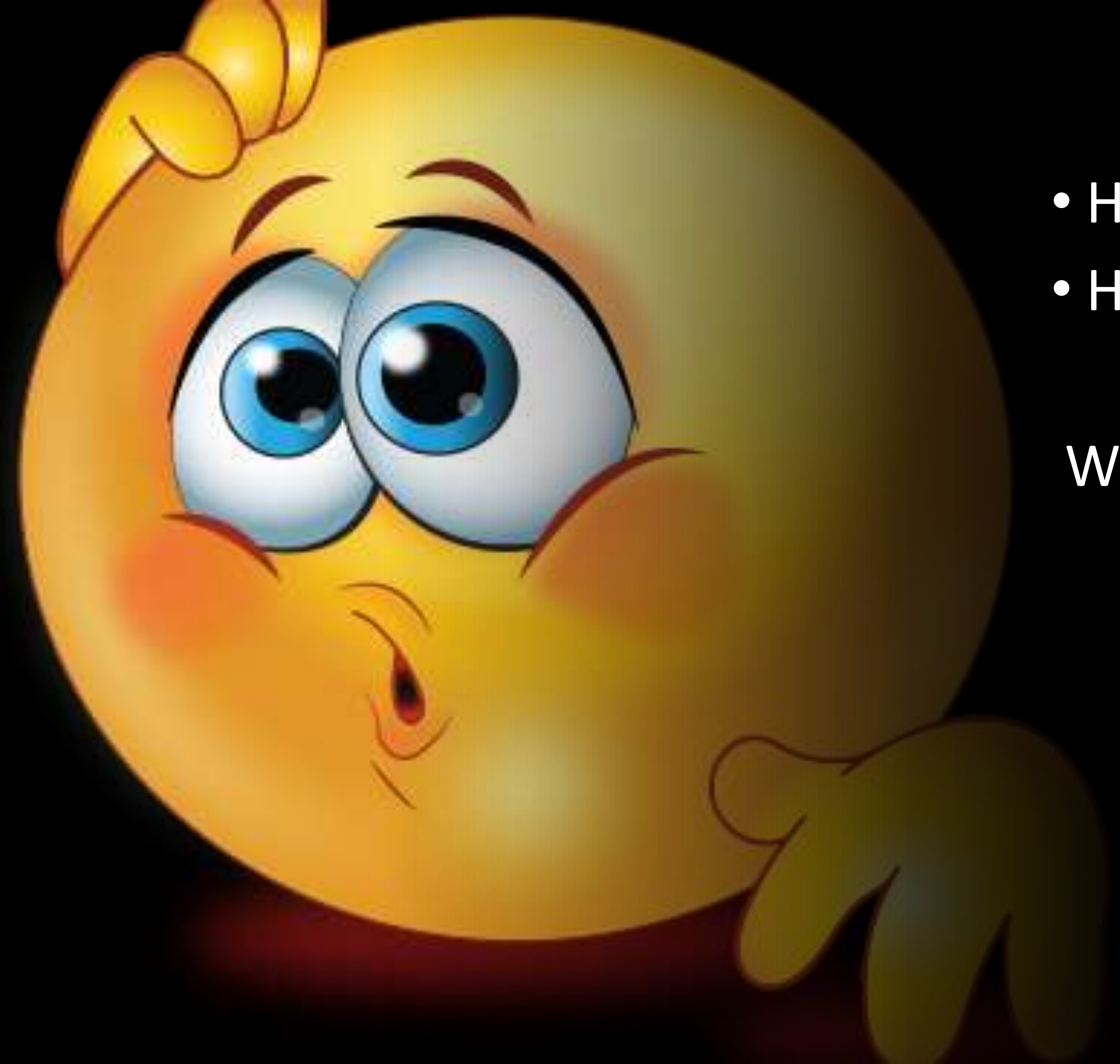
SCRUM is an agile development method which concentrates specifically on how to manage tasks within a team-based development environment. Basically, Scrum is derived from activity that occurs during a rugby match. Scrum believes in empowering the development team and advocates working in small teams (say- 7 to 9 members). Agile and Scrum consist of three roles, and their responsibilities are explained as follows:

# Agile – Scrum Framework



# Agile Concepts –
# I know Agile concepts.

All set for Applied Agile.



- How do I apply my agile skills ?
- How do I Manage ?

Which Tool will help me ?

Not to Worry.

We have Many Tools available in Market.
Trello, Wrike, Teamwork, ClickUp, Asana, Wrike etc.

---

Let’s go for most widely used tool called - JIRA

# What is JIRA?

JIRA is powerful Work management Tool.

Jira is great at helping your team plan and track all the work.

It can help teams work faster, communicate more effectively due to its well-managed workflow, Mapping and issue tracking ability with minimal effort.

# Jira – Default Roles

- Administrators (people who administer a given project).
- Developers (people who work on issues in each project).
- Users (people who log issues on a given project).

## Roles

JIRA enables you to allocate particular people to specific roles in your project. Roles are used when defining other settings, like notifications and permissions.

- Project Lead: Bryan Rollins
- Default Assignee: Unassigned

| Project Roles | Users | Groups |
|---|---|---|
| Administrators | Bryan Rollins, Christina, David | administrators |
| Developers | Penny, Scott | developers |
| Users | Michal Orzechowski | users |

# Jira - Activities

Configure Boards

Configure and Generate Reports

Get Active Sprint Data

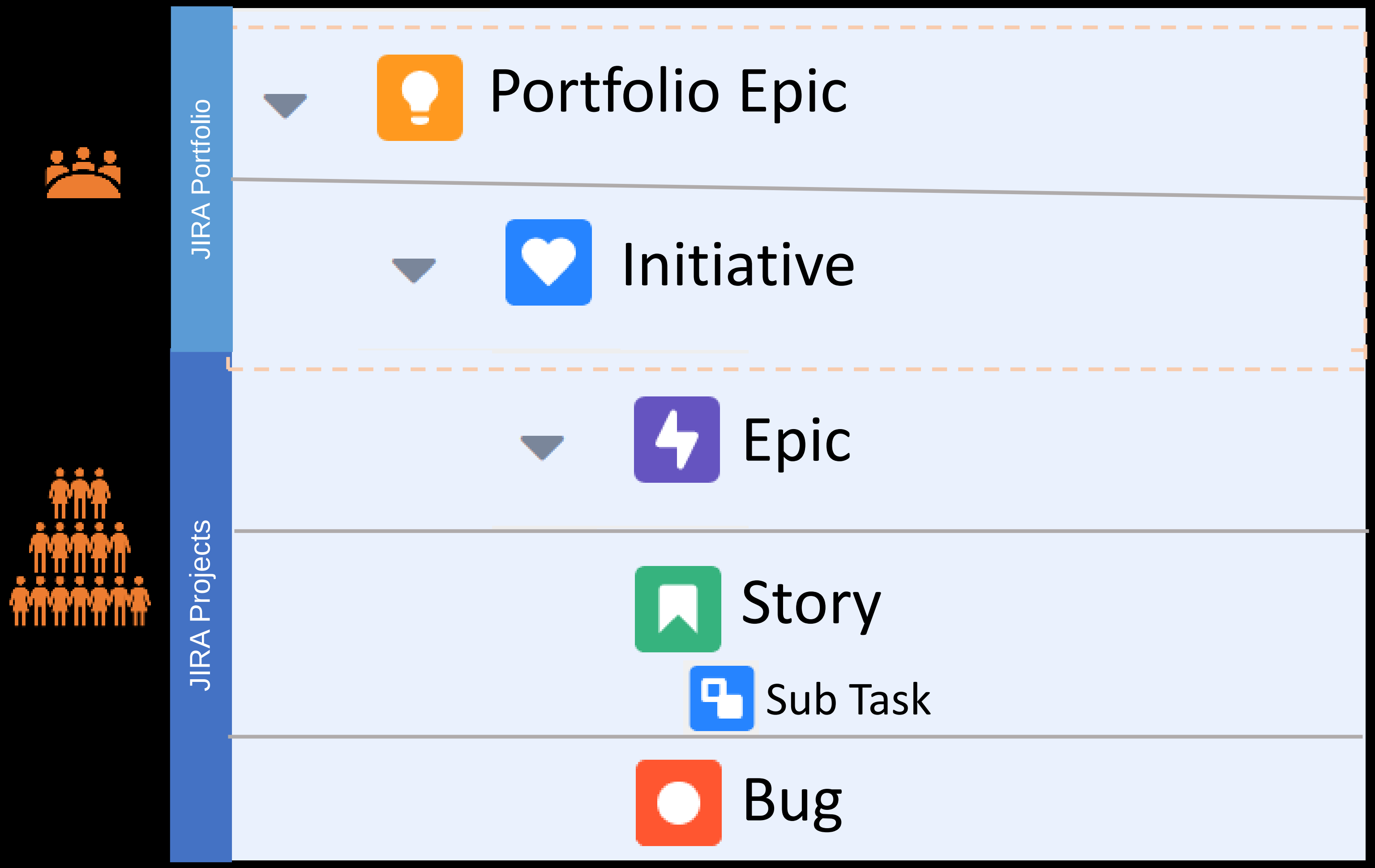
Manage Backlog

Plan and Manage Sprints

Plan and Manage Stories (User Story, Bug)

Setup Project

# JIRA Hierarchy and Story definition




Portfolio Epic and Initiative set by PMO and/or Lead BA/Product Owner

Epics set by Asset BA/ Product Owner

User Story Types:

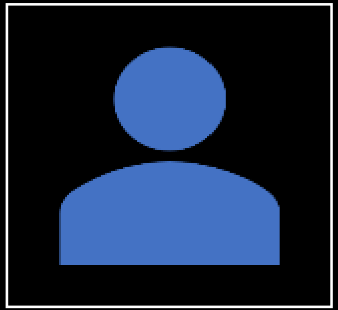
- Development Story [BA]
- Technical Debt Story [Dev Lead]
- Automated Test story [Dev/QA]
- QA Development story [QA]
- SIT Test story [E2E QA Lead]
- UAT Test story [BA]

No Spike or Chore Issue types

# Plan and Manage Sprints

How to create Sprint

How to Start Sprint

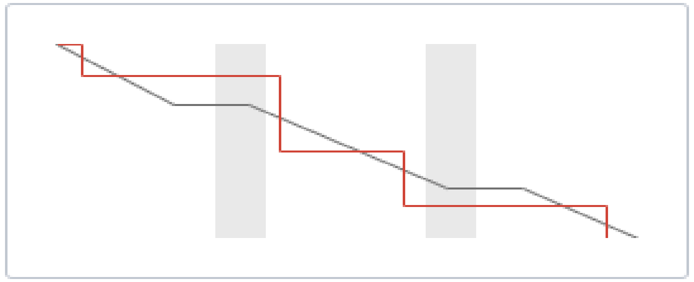


How to Manage Sprint

How to Manage Sprint Backlog

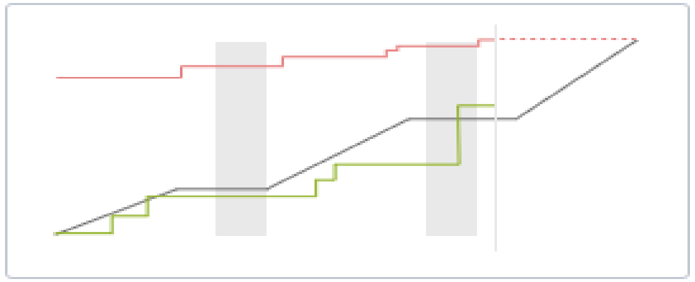
# Jira – Configure and Generate Reports

## Agile



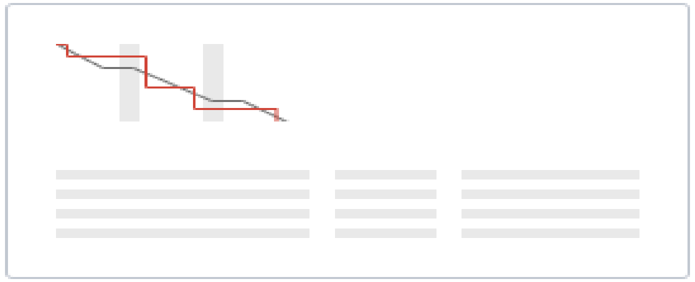
### Burndown Chart

Track the total work remaining and project the likelihood of achieving the sprint goal. This helps your team manage its progress and respond accordingly.



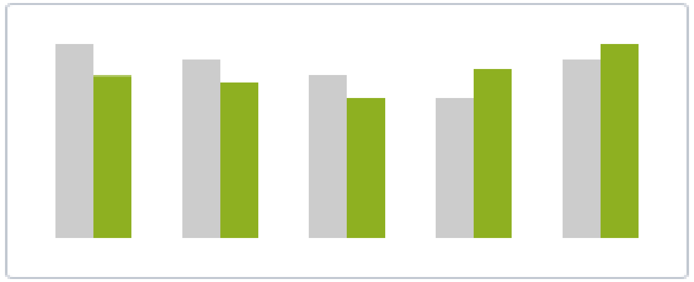
### Burnup Chart

Track the total scope independently from the total work done. This helps your team manage its progress and better understand the effect of scope change.

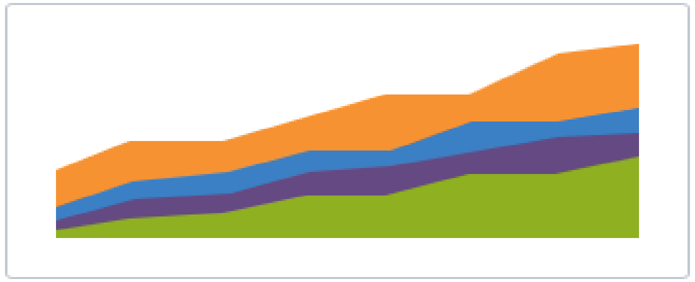


### Sprint Report

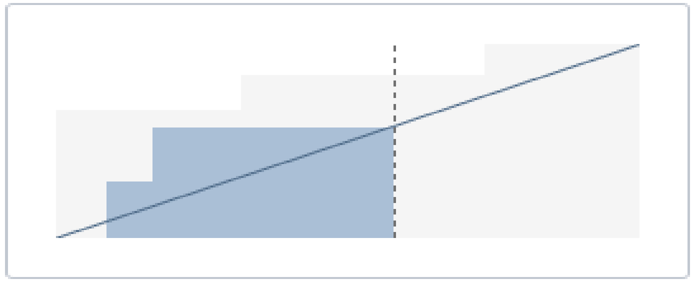
Understand the work completed or pushed back to the backlog in each sprint. This helps you determine if your team is overcommitting or if there is excessive scope creep.



### Velocity Chart



### Cumulative Flow Diagram



### Version Report


Thank You